SLOT 16

LOGIN FLOW & SESSION MANAGEMENT

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

- Login Form & Mock Auth
- Session State Management
- Guarded Routes

1 BỐI CẢNH BÀI HỌC

Trong hầu hết các ứng dụng mobile hiện đại:

người dùng phải **đăng nhập**
hệ thống phải **lưu trạng thái đăng nhập**
một số màn hình **chỉ cho phép truy cập khi đã login**

Ví dụ các ứng dụng:

banking app
social network
e-commerce

Những ứng dụng này đều có **authentication flow**.
Slot 16 giúp sinh viên **xây dựng phiên bản đơn giản của luồng này**.

2 MỤC TIÊU BÀI HỌC

Sau khi hoàn thành bài học, sinh viên có thể:

1 Xây dựng Login Form

Sử dụng:

Form
TextFormField

2 Mô phỏng authentication logic

Thực hiện **Mock Authentication**
không cần server.

3 Quản lý trạng thái đăng nhập

Lưu **session state** trong app.

4 Bảo vệ các màn hình cần login

Triển khai **Guarded Routes**.

3 KHÁI NIỆM TRUNG TÂM CỦA BÀI HỌC

Slot 16 tập trung vào 3 khái niệm quan trọng:

Khái niệm	Ý nghĩa
Login Form	giao diện nhập thông tin đăng nhập
Session State	trạng thái đăng nhập của user
Guarded Routes	bảo vệ screen cần login

4 LOGIN FORM & MOCK AUTHENTICATION

Login Form là gì?

Login Form là màn hình cho phép người dùng nhập:

username
password

PhucPT10@fpt.edu.vn

Ví dụ UI:

```
Username: [________]

Password: [________]

[ LOGIN ]
```

Trong Flutter, Login Form thường sử dụng:

- **Form**
- **TextFormField**
- **ElevatedButton**

Mock Authentication là gì?

Mock authentication là:
mô phỏng logic đăng nhập mà **không cần server thật**.
Ví dụ logic đơn giản:

```
username = admin
password = 1234
```

Nếu đúng:

```
login success
```

Nếu sai:

```
login failed
```

Mục đích của Mock Auth

Giúp sinh viên:

- hiểu **luồng đăng nhập**
- test UI
- phát triển app trước khi có backend.

5 SESSION STATE MANAGEMENT

Session là gì?

Session là **trạng thái đăng nhập của user trong ứng dụng**.
Ví dụ:

```
isLoggedIn = true
```

hoặc

```
isLoggedIn = false
```

Các trạng thái session cơ bản

State	Ý nghĩa
Logged Out	chưa đăng nhập
Logged In	đã đăng nhập

Session được lưu ở đâu?

Trong bài học này có 2 cách:

1 Memory State

Lưu session trong biến.
Ví dụ:

```
bool isLoggedIn = false;
```

2 Local Storage (Optional)

Lưu session bằng:

- **shared_preferences**

Điều này giúp:

PhucPT10@fpt.edu.vn

app restart vẫn nhớ user login.

6 GUARDED ROUTES

Guarded Route là gì?

Guarded route là:
màn hình chỉ cho phép truy cập khi user đã login.
Ví dụ:

Screen	Yêu cầu
Dashboard	phải login
Profile	phải login
Settings	phải login

Logic Guarded Route

Pseudo code:

```
if (isLoggedIn == true)
    open Dashboard
else
    redirect to Login
```

Ví dụ thực tế

User mở:

```
Profile Screen
```

Nếu chưa login:

```
redirect → Login Screen
```

7 AUTHENTICATION FLOW TRONG APP

Luồng hoạt động chuẩn của ứng dụng:

```
App Start
   ↓
Check Session
   ↓
isLoggedIn ?
   ↓
YES → Dashboard
NO  → Login Screen
```

8 FLOW ĐĂNG NHẬP TRONG DEMO APP

Luồng đăng nhập:

```
User nhập username/password
      ↓
Submit login
      ↓
Check credential
      ↓
Login success
      ↓
Save session
      ↓
Navigate to Home
```

9 FLOW LOGOUT

Luồng logout:

PhucPT10@fpt.edu.vn

```
User click Logout
      ↓
Clear session
      ↓
Navigate to Login
```

10 MINI DEMO APP CỦA SLOT 16

Ứng dụng demo gồm 3 màn hình:

Screen	Chức năng
Login Screen	đăng nhập
Home Screen	màn hình chính
Profile Screen	màn hình cần login

Flow hoạt động

```
Login Screen
     ↓
Login success
     ↓
Home Screen
     ↓
Profile Screen
```

Nếu logout:

```
Profile
↓
Logout
↓
Login Screen
```

11 KIẾN THỨC LIÊN QUAN CÁC SLOT TRƯỚC

Slot 16 liên kết với các kiến thức đã học:

Slot	Kiến thức
Slot 12	Form UI
Slot 13	API data
Slot 15	Data persistence

12 TƯ DUY KIẾN TRÚC QUAN TRỌNG

Sinh viên cần hiểu 3 nguyên tắc:

1 Authentication không chỉ là UI

nó là **luồng hệ thống**.

2 Session quyết định quyền truy cập

```
session → access control
```

3 Route protection là bắt buộc

mọi ứng dụng production đều có.

🎓 KẾT LUẬN SLOT 16

Sau bài học này sinh viên sẽ hiểu:

1 Cách xây dựng Login Form

PhucPT10@fpt.edu.vn

2 Cách mô phỏng authentication

3 Cách quản lý session

4 Cách bảo vệ các màn hình cần login

TÀI LIỆU THAM KHẢO

Flutter & Storage

Flutter Official Documentation
https://docs.flutter.dev

JSON trong Dart

Dart JSON Guide
https://dart.dev/guides/json

Flutter List Rendering

Flutter ListView Tutorial
https://docs.flutter.dev/cookbook/lists

JSON Serialization Guide

Flutter JSON serialization
https://docs.flutter.dev/data-and-backend/serialization/json

Video học Flutter dễ hiểu

Flutter Official YouTube
https://youtube.com/flutterdev

PhucPT10@fpt.edu.vn

II. TRỌNG TÂM CỐT LÕI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

1. AUTHENTICATION FLOW

Authentication Flow là gì?

Authentication flow là **luồng xử lý đăng nhập trong ứng dụng**.
Luồng này quyết định:

- khi nào user phải login
- khi nào user được truy cập app
- khi nào user bị logout

Luồng Authentication cơ bản

```
App Start
   ↓
Check Session
   ↓
User Logged In ?
   ↓
YES → Home Screen
NO  → Login Screen
```

Luồng Login

```
User nhập username/password
        ↓
Submit Login
        ↓
Validate Credentials
        ↓
Login Success
        ↓
Save Session
        ↓
Navigate → Home
```

Luồng Logout

```
User click Logout
      ↓
Clear Session
      ↓
Redirect → Login Screen
```

Ý nghĩa trong thực tế

Authentication Flow giúp:

- bảo vệ dữ liệu người dùng
- kiểm soát quyền truy cập
- quản lý trạng thái đăng nhập

Đây là kiến trúc có trong **hầu hết mọi ứng dụng production**.

2 LOGIN – SESSION – LOGOUT

Sinh viên cần hiểu **ba khái niệm trung tâm** của authentication.

LOGIN

Login là quá trình:

PhucPT10@fpt.edu.vn

```
User nhập thông tin
→ hệ thống kiểm tra
→ cho phép truy cập
```

Thông tin thường gồm:

- username / email
- password

Trong Flutter, Login Form thường sử dụng:

- **Form**
- **TextFormField**
- **ElevatedButton**

SESSION

Session là **trạng thái đăng nhập của người dùng trong ứng dụng**.

Ví dụ:

```
isLoggedIn = true
```

hoặc

```
isLoggedIn = false
```

Session quyết định:

- user được truy cập screen nào
- user có cần login lại hay không

Các trạng thái session

State	Ý nghĩa
Logged Out	chưa đăng nhập
Logged In	đã đăng nhập

LOGOUT

Logout là quá trình:

```
xóa trạng thái đăng nhập
```

Ví dụ:

```
isLoggedIn = false
```

Sau đó:

```
redirect → Login Screen
```

3 SESSION STATE MANAGEMENT

Session State Management là quá trình:

```
lưu và kiểm soát trạng thái đăng nhập
```

Cách lưu session

1 Memory State

```
bool isLoggedIn = false;
```

Ưu điểm:

- đơn giản
- dễ hiểu

Nhược điểm:

- mất khi app restart.

2 Local Storage (nâng cao)

Dùng plugin:

- **shared_preferences**

Giúp:

```
app restart vẫn nhớ user login
```


PhucPT10@fpt.edu.vn

Khi mở app cần làm gì?

Ứng dụng phải:

```
Check Session
```

Pseudo logic:

```
if (isLoggedIn)
    open Home
else
    open Login
```

4 ROUTE GUARD

Route Guard là cơ chế:

```
kiểm soát truy cập screen
```

Guarded Route là gì?

Guarded route là screen:

```
chỉ cho phép truy cập khi user đã login
```

Ví dụ:

Screen	Protection
Dashboard	Guarded
Profile	Guarded
Settings	Guarded

Logic Route Guard

```
if (isLoggedIn == false)
    redirect → Login
```

Ví dụ thực tế

User cố mở:

```
Profile Screen
```

Nếu chưa login:

```
redirect → Login Screen
```

KỸ NĂNG CẦN ĐẠT

Sau bài học, sinh viên phải **thực hiện được các kỹ năng thực tế**.

1 XÂY DỰNG LOGIN SCREEN

Sinh viên có thể tạo giao diện đăng nhập gồm:

```
Username input
Password input
Login button
```

Dùng widget:

TextFormField

2 XỬ LÝ SUBMIT FORM

Khi user nhấn login:

```
validate input
```

Sau đó:

```
check credential
```

Ví dụ mock auth:

```
username = admin
password = 1234
```


PhucPT10@fpt.edu.vn

3 LƯU SESSION

Sau khi login thành công:

```
save session
```

Ví dụ:

```
isLoggedIn = true;
```

4 REDIRECT USER SAU LOGIN

Sau khi login:

```
navigate → Home Screen
```

Dùng Navigator trong Flutter.

5 BẢO VỆ ROUTE

Khi user mở screen:

```
check session
```

Nếu:

```
not logged in
```

→ redirect về login.

⚠ NHỮNG HIỂU SAI PHỔ BIẾN

Sinh viên mới học Flutter thường hiểu sai về authentication.

✕ HIỂU SAI 1 — LOGIN CHỈ LÀ UI

Nhiều sinh viên nghĩ:

```
login = form + button
```

Thực tế:

```
login = authentication flow
```

Nó bao gồm:

- form
- validation
- session
- navigation.

✕ HIỂU SAI 2 — KHÔNG QUẢN LÝ SESSION

Một số app student làm:

```
login → navigate
```

nhưng:

```
không lưu session
```

Kết quả:

app restart → mất login.

✕ HIỂU SAI 3 — KHÔNG KIỂM TRA LOGIN KHI NAVIGATE

Sinh viên thường:

```
Navigator.push(Profile)
```

nhưng:

```
không check session
```

Kết quả:

user chưa login vẫn mở được screen.

🎓 BÀI HỌC QUAN TRỌNG

Sinh viên cần ghi nhớ **3 nguyên tắc cốt lõi**.

PhucPT10@fpt.edu.vn

1 LOGIN LÀ MỘT FLOW

```
UI
→ validation
→ authentication
→ session
→ navigation
```

2 SESSION QUYẾT ĐỊNH QUYỀN TRUY CẬP

```
session = access control
```

3 ROUTE GUARD BẢO VỆ ỨNG DỤNG

```
protected screen → require login
```

•

.

PhucPT10@fpt.edu.vn

III. ỨNG DỤNG THỰC TIỄN

1 AUTHENTICATION TRONG ỨNG DỤNG THẬT

Trong thực tế, gần như **mọi ứng dụng mobile hiện đại đều có hệ thống authentication**.
Mục đích chính:

xác định **người dùng là ai**
bảo vệ **dữ liệu cá nhân**
quản lý **quyền truy cập**

Authentication thường bao gồm:

```
Login
Session
Authorization
Logout
```

2 CÁC LOẠI ỨNG DỤNG CẦN AUTHENTICATION

E-commerce App

Ví dụ:

Shopee
Lazada

User phải login để:

xem đơn hàng
lưu địa chỉ giao hàng
thanh toán.

Banking App

Ví dụ:

Vietcombank Digibank

Login là **bắt buộc** để:

xem số dư
chuyển tiền
quản lý tài khoản.

Đây là trường hợp **authentication + security rất nghiêm ngặt**.

Social Network

Ví dụ:

Facebook
Instagram

User login để:

đăng bài
xem feed
gửi tin nhắn.

Learning App

Ví dụ:

Duolingo

Login giúp:

lưu tiến trình học
đồng bộ dữ liệu.

3 TÌNH HUỐNG THỰC TẾ TRONG APP

Các ứng dụng thật thường gặp **những tình huống phổ biến sau**.

PhucPT10@fpt.edu.vn

TÌNH HUỐNG 1 – APP CẦN LOGIN TRƯỚC KHI SỬ DỤNG

Một số ứng dụng **không cho phép truy cập nếu chưa login**.
Ví dụ:

```
App Start
↓
Login Screen
↓
Home Screen
```

User phải login để:
    xem dữ liệu cá nhân
    truy cập chức năng chính.

Ví dụ thực tế

Ứng dụng task manager.
User phải login để:

```
view tasks
create tasks
update tasks
```

Điều này giúp:
    mỗi user có dữ liệu riêng.

TÌNH HUỐNG 2 – BẢO VỆ DỮ LIỆU NGƯỜI DÙNG

Một số màn hình cần được **bảo vệ bằng authentication**.
Ví dụ:

Screen	Yêu cầu
Home	có thể public
Profile	phải login
Settings	phải login

Logic:

```
if (isLoggedIn == false)
    redirect → Login Screen
```

Đây chính là **Guarded Routes**.

TÌNH HUỐNG 3 – SESSION PERSISTENCE

Một trong những yêu cầu phổ biến của user:
không muốn login lại mỗi lần mở app.
Nếu session không được lưu:

```
App Restart
↓
User phải login lại
```

Trải nghiệm này **rất tệ**.

Giải pháp

Lưu session bằng:
    shared_preferences
Khi app mở:

```
Check session
↓
Nếu logged in → Home
Nếu không → Login
```

4 CASE STUDY – TODO APP CÓ ACCOUNT

Để hiểu rõ hơn, hãy phân tích một **Todo App có hệ thống account**.

PhucPT10@fpt.edu.vn

Cấu trúc hệ thống

Ứng dụng có các màn hình:

Screen	Chức năng
Login	đăng nhập
Home	danh sách tasks
Profile	thông tin user

Flow hoạt động

```
App Start
    ↓
Check Session
    ↓
User Logged In ?
    ↓
YES → Home
NO  → Login
```

Flow đăng nhập

```
Login Screen
      ↓
User nhập credential
      ↓
Authentication success
      ↓
Save session
      ↓
Navigate → Home
```

Flow sử dụng ứng dụng

Sau khi login:

```
User Access Tasks
↓
Create task
↓
Update task
↓
Delete task
```

Mỗi user sẽ có:

```
personal task list
```

Flow logout

```
User click Logout
      ↓
Clear session
      ↓
Redirect → Login Screen
```

5 KIẾN TRÚC AUTHENTICATION CỦA APP

Trong Todo App, authentication ảnh hưởng đến **toàn bộ ứng dụng**.

```
Authentication Layer
        ↓
Session State
```


PhucPT10@fpt.edu.vn

```
        ↓
Route Guard
        ↓
App Screens
```

6 LỢI ÍCH CỦA AUTHENTICATION

Authentication mang lại nhiều lợi ích:

1 Bảo mật dữ liệu

Mỗi user chỉ thấy dữ liệu của mình.

2 Cá nhân hóa ứng dụng

Ví dụ:

```
Hello John
```

3 Đồng bộ dữ liệu

User có thể:

```
login trên nhiều thiết bị
```

7 BÀI HỌC QUAN TRỌNG CHO SINH VIÊN

Sinh viên cần hiểu rằng:

1 Authentication không chỉ là Login Screen

Nó bao gồm:

```
login
session
route guard
logout
```

2 Session là trung tâm của authentication

```
session = access control
```

3 Guarded routes bảo vệ ứng dụng

```
protected screens require login
```

🎓 KẾT LUẬN

Authentication là một trong những **tính năng quan trọng nhất của ứng dụng mobile**.

Sinh viên cần hiểu:

- cách xây dựng login flow
- cách quản lý session
- cách bảo vệ route.

PhucPT10@fpt.edu.vn

IV. MỘT SỐ NỘI DUNG CHI TIẾT

1 BẢN CHẤT CỦA AUTHENTICATION

Authentication là gì?

Authentication là quá trình:

```
xác thực danh tính của người dùng
```

Hệ thống cần trả lời câu hỏi:

```
User này có thực sự là người mà họ khai báo không?
```

Ví dụ:

User nhập:

```
username = admin
password = 1234
```

Hệ thống kiểm tra:

```
credential đúng hay sai
```

Nếu đúng:

```
authentication success
```

Nếu sai:

```
authentication failed
```

Authentication vs Authorization

Sinh viên thường nhầm lẫn hai khái niệm này.

Khái niệm	Ý nghĩa
Authentication	xác thực người dùng
Authorization	phân quyền truy cập

Ví dụ:

```
Login → Authentication
Admin page → Authorization
```

PHẦN A
LOGIN FLOW

Login Flow là gì?

Login Flow là **chuỗi các bước mà hệ thống thực hiện để xác thực người dùng**.

Quy trình chuẩn:

```
User Input
    ↓
Validate Input
    ↓
Verify Credentials
    ↓
Create Session
    ↓
Access Application
```

1 USER INPUT

Người dùng nhập thông tin:

- username / email
- password

Trong Flutter UI thường dùng:

- **Form**
- **TextFormField**

2 VALIDATE INPUT

Ứng dụng cần kiểm tra dữ liệu trước khi gửi login.

PhucPT10@fpt.edu.vn

Ví dụ:
`username không được rỗng`
`password tối thiểu 6 ký tự`
Mục tiêu:
- tránh lỗi
- giảm request không hợp lệ.

3 VERIFY CREDENTIALS
Hệ thống kiểm tra:
`username/password`
Có hai trường hợp:
Mock authentication
logic nằm trong app.
Ví dụ:
`admin / 1234`

API authentication
App gửi request đến server.
`POST /login`
Server kiểm tra database.

4 CREATE SESSION
Sau khi login thành công:
hệ thống tạo **session**.
Session giúp ứng dụng biết:
`user đã login`

PHẦN B
SESSION MANAGEMENT

Session là gì?
Session là **trạng thái đăng nhập của người dùng trong ứng dụng**.
Ví dụ:
`isLoggedIn = true`
Session giúp ứng dụng trả lời:
`User đã login chưa?`

Vai trò của session
Session quyết định:
- user có thể truy cập screen nào
- user có cần login lại hay không.

CÁC CÁCH QUẢN LÝ SESSION

1 IN-MEMORY STATE
Session được lưu trong RAM.
Ví dụ:
`bool isLoggedIn = true;`
Ưu điểm:
- đơn giản
- dễ triển khai.

Nhược điểm:
`app restart → mất session`

PhucPT10@fpt.edu.vn

2 LOCAL STORAGE

Session được lưu trong thiết bị.
Flutter thường dùng:

shared_preferences

Ví dụ:

```
isLoggedIn = true
```

được lưu vào storage.
Khi app mở lại:

```
load session
```

3 TOKEN-BASED SESSION

Đây là cách phổ biến trong **ứng dụng production**.
Sau khi login thành công:
server trả về:

```
access token
```

Ví dụ:

```
{
 "token": "abc123xyz"
}
```

App lưu token và gửi token trong mọi request API.

PHẦN C
ROUTE PROTECTION

Route Protection là gì?

Route protection là cơ chế:

```
ngăn user chưa login truy cập protected screen
```

Ví dụ:

Screen	Protection
Login	public
Home	protected
Profile	protected

Route Guard hoạt động thế nào?

Pseudo logic:

```
if (isLoggedIn == false)
    redirect → Login
```

Ví dụ trong Flutter

User cố mở:

```
ProfileScreen
```

App kiểm tra:

```
session
```

Nếu chưa login:

```
redirect → LoginScreen
```

PHẦN D
KIẾN TRÚC AUTHENTICATION TRONG APP LỚN

Trong ứng dụng production, authentication phức tạp hơn nhiều.
Kiến trúc thường gồm:

PhucPT10@fpt.edu.vn

```
Mobile App
      ↓
Authentication API
      ↓
Token Management
      ↓
Secure Storage
```

1 API AUTHENTICATION

Ứng dụng mobile gửi request:

```
POST /login
```

Server kiểm tra:

- database
- password hash.

Nếu đúng:

server trả về token.

2 TOKEN MANAGEMENT

Token được dùng để xác thực các request tiếp theo.

Ví dụ:

```
GET /profile
Authorization: Bearer TOKEN
```

Server biết request này đến từ user đã login.

3 REFRESH TOKEN

Access token thường có thời hạn.

Ví dụ:

```
expires in 1 hour
```

Khi hết hạn:

app dùng **refresh token** để lấy token mới.

4 SECURE STORAGE

Token không được lưu ở nơi không an toàn.

Flutter thường dùng:

- **flutter_secure_storage**

Điều này giúp:

```
token được mã hóa
```

5 AUTH ARCHITECTURE TỔNG QUÁT

Authentication trong production app thường có kiến trúc:

```
Login UI
   ↓
Auth Service
   ↓
API Authentication
   ↓
Token Storage
   ↓
Session State
   ↓
Route Guard
   ↓
Protected Screens
```


PhucPT10@fpt.edu.vn

🎓 BÀI HỌC QUAN TRỌNG

Sinh viên cần hiểu **3 nguyên tắc cốt lõi** của authentication architecture.

1 Authentication là một hệ thống

không chỉ là Login Form.

2 Session là trung tâm của quyền truy cập

```
session = access control
```

3 Route guard bảo vệ dữ liệu người dùng

```
protected screens require authentication
```


PhucPT10@fpt.edu.vn

HƯỚNG DẪN ÁP DỤNG

I. LOGIN FORM & MOCK AUTH (FLUTTER)

1. MỤC TIÊU BÀI HỌC

Sau khi hoàn thành bài này sinh viên có thể:

- tạo **Login Screen**
- xử lý **form input**
- kiểm tra **username và password**
- mô phỏng **authentication logic**
- điều hướng sang **Home Screen sau login**

2. KHÁI NIỆM MOCK AUTH

Trong hệ thống thực tế:

```
App → API Server → Database
```

Nhưng khi học Flutter cơ bản ta **chưa cần backend**.
Vì vậy ta dùng **Mock Authentication**.
Mock Authentication nghĩa là:

```
tạo dữ liệu user giả trong code
và kiểm tra login dựa trên dữ liệu này
```

Ví dụ user giả:

```
username: admin
password: 123456
```

3. LUỒNG LOGIN CƠ BẢN

Luồng xử lý:

```
User nhập username + password
        ↓
Validate input
        ↓
Check credentials
        ↓
Nếu đúng → Login success
Nếu sai → Hiển thị lỗi
```

4. BƯỚC 1 – TẠO PROJECT FLUTTER

Tạo project mới

```
flutter create login_mock_app
```

Mở project bằng:

```
VSCode
Android Studio
```

5. BƯỚC 2 – TẠO LOGIN SCREEN

Tạo file:

```
lib/screens/login_screen.dart
```

Code khung Login Screen

```
import 'package:flutter/material.dart';

class LoginScreen extends StatefulWidget {
  @override
  _LoginScreenState createState() => _LoginScreenState();
}
```


PhucPT10@fpt.edu.vn

Giải thích

Thành phần	Ý nghĩa
StatefulWidget	vì login form có state
_LoginScreenState	nơi xử lý logic

6. BƯỚC 3 – TẠO CONTROLLERS CHO INPUT

Thêm vào class state:

```
class _LoginScreenState extends State<LoginScreen> {

  final TextEditingController usernameController =
TextEditingController();
  final TextEditingController passwordController =
TextEditingController();

}
```

Giải thích

Controller giúp:

```
đọc dữ liệu user nhập vào TextField
```

Ví dụ:

```
usernameController.text
```

7. BƯỚC 4 – THIẾT KẾ LOGIN FORM UI

Thêm hàm build:

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text("Login"),
    ),
    body: Padding(
      padding: EdgeInsets.all(20),
      child: Column(
        children: [

          TextField(
            controller: usernameController,
            decoration: InputDecoration(
              labelText: "Username",
            ),
          ),

          SizedBox(height: 20),

          TextField(
            controller: passwordController,
            obscureText: true,
            decoration: InputDecoration(
              labelText: "Password",
            ),
          ),

          SizedBox(height: 20),
```


PhucPT10@fpt.edu.vn

```
          ElevatedButton(
            onPressed: handleLogin,
            child: Text("Login"),
          )

        ],
      ),
    ),
  );
}
```

8. GIẢI THÍCH UI

UI gồm:

```
Username TextField
Password TextField
Login Button
```

obscureText

```
obscureText: true
```

Giúp **ẩn password**

Ví dụ:

```
123456 → ******
```

9. BƯỚC 5 – TẠO MOCK AUTH LOGIC

Thêm hàm:

```
void handleLogin() {

  String username = usernameController.text;
  String password = passwordController.text;

  if(username == "admin" && password == "123456"){
    showSuccess();
  } else {
    showError();
  }

}
```

Giải thích

Ta tạo **user giả**

```
admin / 123456
```

Hệ thống kiểm tra:

```
username == admin
password == 123456
```

10. BƯỚC 6 – HIỂN THỊ LOGIN SUCCESS

```
void showSuccess() {
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(
      content: Text("Login success"),
    ),
  );
}
```


PhucPT10@fpt.edu.vn

11. BƯỚC 7 – HIỂN THỊ LOGIN ERROR

```
void showError() {
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(
      content: Text("Invalid username or password"),
    ),
  );
}
```

12. BƯỚC 8 – KIỂM TRA INPUT VALIDATION

Sinh viên thường quên bước này.
Thêm kiểm tra:

```
if(username.isEmpty || password.isEmpty){
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text("Please fill all fields")),
  );
  return;
}
```

Luồng đầy đủ:

```
check empty
→ check credential
→ success hoặc error
```

13. BƯỚC 9 – CHUYỂN SANG HOME SCREEN

Tạo file:

```
home_screen.dart
```

Code:

```
class HomeScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Home")),
      body: Center(
        child: Text("Welcome to Gallery App"),
      ),
    );
  }
}
```

Update handleLogin

```
if(username == "admin" && password == "123456"){

  Navigator.pushReplacement(
    context,
    MaterialPageRoute(builder: (context) => HomeScreen()),
  );

}
```

14. TEST KẾT QUẢ

Chạy app:

```
flutter run
```

Test:

PhucPT10@fpt.edu.vn

Case 1

```
admin
123456
```

Kết quả:

```
Login success
→ chuyển sang Home
```

Case 2

```
admin
111111
```

Kết quả:

```
Invalid username or password
```

15. LỖI SINH VIÊN THƯỜNG GẶP

Lỗi 1

Không dùng controller

```
TextField không đọc được dữ liệu
```

Lỗi 2

Quên obscureText

```
password hiển thị rõ
```

Lỗi 3

Không validate input

```
login khi field rỗng
```

16. BEST PRACTICES

1. Validate trước khi login

```
if(field empty) → báo lỗi
```

2. Không hardcode credential trong production

Mock auth chỉ dùng cho:

```
learning
prototype
testing UI
```

3. Tách logic thành service

Ví dụ:

```
auth_service.dart
```

17. TÓM TẮT CÁC BƯỚC CHÍNH

Quy trình thực hiện:

```
B1 tạo LoginScreen
B2 tạo TextEditingController
B3 tạo TextField
B4 tạo Login button
B5 viết handleLogin()
B6 kiểm tra credential
B7 hiển thị Snackbar
B8 navigate HomeScreen
```

18. MINI BÀI TẬP CHO SINH VIÊN

Hãy cải tiến Login App:

PhucPT10@fpt.edu.vn

1. thêm **Register Screen**
2. thêm **Logout button**
3. lưu **session bằng SharedPreferences**

PhucPT10@fpt.edu.vn

II. SESSION STATE MANAGEMENT (FLUTTER)

1. MỤC TIÊU BÀI HỌC

Sau phần này sinh viên có thể:

- hiểu khái niệm **session**
- lưu trạng thái **user login**
- kiểm tra session khi mở app
- tự động redirect screen
- xử lý **logout**

2. SESSION LÀ GÌ

Session là **trạng thái đăng nhập của người dùng**.

Ví dụ:

```
User login thành công
→ app cần nhớ user đã login
```

Nếu không có session:

```
User phải login lại mỗi lần mở app
```

Điều này **rất tệ cho trải nghiệm người dùng**.

3. LUỒNG SESSION TRONG APP

Luồng hoạt động chuẩn:

```
App start
    ↓
Check session
    ↓
Nếu có session → HomeScreen
Nếu chưa login → LoginScreen
```

4. CÁCH LƯU SESSION

Trong Flutter có nhiều cách:

Phương pháp	Mục đích
in-memory	chỉ tồn tại khi app chạy
SharedPreferences	lưu local
Hive	database local
SecureStorage	bảo mật cao

Trong bài học này ta dùng:

```
SharedPreferences
```

5. BƯỚC 1 – CÀI PACKAGE

Mở file:

```
pubspec.yaml
```

Thêm:

```
dependencies:
  shared_preferences: ^2.2.2
```

Chạy:

```
flutter pub get
```

6. BƯỚC 2 – IMPORT PACKAGE

Trong file login:

```
import 'package:shared_preferences/shared_preferences.dart';
```

7. BƯỚC 3 – LƯU SESSION SAU LOGIN

Khi login thành công ta lưu trạng thái.

Ví dụ:

```
Future<void> saveSession() async {

  final prefs = await SharedPreferences.getInstance();

  await prefs.setBool("isLoggedIn", true);

}
```

Giải thích

Ta lưu một giá trị:

```
key = isLoggedIn
value = true
```

Nghĩa là:

```
user đã login
```

8. UPDATE HANDLE LOGIN

Sau khi login success:

```
void handleLogin() async {

  String username = usernameController.text;
  String password = passwordController.text;

  if(username == "admin" && password == "123456"){

    await saveSession();

    Navigator.pushReplacement(
      context,
      MaterialPageRoute(builder: (_) => HomeScreen()),
    );

  }else{

    showError();

  }

}
```

9. BƯỚC 4 – KIỂM TRA SESSION KHI MỞ APP

Ta cần kiểm tra session khi app start.

Mở file:

```
main.dart
```

Code kiểm tra session

```
Future<bool> checkSession() async {

  final prefs = await SharedPreferences.getInstance();

  return prefs.getBool("isLoggedIn") ?? false;

}
```

Giải thích

```
getBool("isLoggedIn")
```

Nếu chưa có giá trị:

```
?? false
```

10. CHỌN SCREEN KHI APP START

Trong `main.dart`:

```
void main() async {

  WidgetsFlutterBinding.ensureInitialized();

  final prefs = await SharedPreferences.getInstance();

  bool isLoggedIn = prefs.getBool("isLoggedIn") ?? false;

  runApp(MyApp(isLoggedIn));

}
```

11. UPDATE MYAPP

```
class MyApp extends StatelessWidget {

  final bool isLoggedIn;

  MyApp(this.isLoggedIn);

  @override
  Widget build(BuildContext context) {
    return MaterialApp(

      home: isLoggedIn
          ? HomeScreen()
          : LoginScreen(),

    );
  }
}
```

12. KẾT QUẢ

Luồng hoạt động:

```
App start
↓
check isLoggedIn
↓
true  → HomeScreen
false → LoginScreen
```

13. BƯỚC 5 – IMPLEMENT LOGOUT

Trong HomeScreen tạo button:

```
ElevatedButton(
  onPressed: logout,
  child: Text("Logout"),
)
```

Hàm logout

```
Future<void> logout() async {

  final prefs = await SharedPreferences.getInstance();

  await prefs.remove("isLoggedIn");

  Navigator.pushReplacement(
    context,
    MaterialPageRoute(builder: (_) => LoginScreen()),
  );

}
```

14. GIẢI THÍCH LOGOUT

Ta xóa session:

```
remove("isLoggedIn")
```

Sau đó:

```
navigate → LoginScreen
```

15. TEST KẾT QUẢ

Test theo các bước.

Test 1

```
Login
→ close app
→ open lại
```

Kết quả:

```
app vào HomeScreen
```

Test 2

```
Logout
→ close app
→ open lại
```

Kết quả:

```
app vào LoginScreen
```

16. LỖI SINH VIÊN THƯỜNG GẶP

Lỗi 1

Quên await

Sai:

```
saveSession();
```

Đúng:

```
await saveSession();
```

Lỗi 2

Không gọi

```
WidgetsFlutterBinding.ensureInitialized();
```

Trong main.

Lỗi 3

Logout không clear session

Kết quả:

PhucPT10@fpt.edu.vn

```
app tự login lại
```

17. BEST PRACTICES

1. Session phải có single source

Ví dụ:

```
SharedPreferences
```

2. Không lưu password

Sai:

```
password: 123456
```

Đúng:

```
chỉ lưu token / trạng thái login
```

3. Dùng AuthService

Trong app lớn:

```
auth_service.dart
```

18. TÓM TẮT CÁC BƯỚC

Quy trình Session Management:

```
B1 login thành công
B2 lưu session
B3 app start → check session
B4 redirect screen
B5 logout → clear session
```

19. MINI BÀI TẬP CHO SINH VIÊN

Sinh viên hãy cải tiến app:

1 lưu **username trong session**

2 hiển thị:

```
Welcome admin
```

3 tạo **Guarded Route**

```
user chưa login → không vào Home
```


PhucPT10@fpt.edu.vn

III. GUARDED ROUTES (ROUTE PROTECTION)

1. MỤC TIÊU BÀI HỌC

Sau phần này sinh viên có thể:

• hiểu khái niệm **Guarded Route**
• kiểm soát quyền truy cập screen
• chặn user chưa login
• kết hợp với **Session State Management**

2. VẤN ĐỀ THỰC TẾ

Nếu ứng dụng **không có Guarded Route** thì có thể xảy ra tình huống:

```
User chưa login
↓
vẫn truy cập được HomeScreen
```

Ví dụ:

```
Login → Home
User logout
Nhưng vẫn mở lại Home bằng navigation
```

Điều này gây:

```
vi phạm bảo mật
lộ dữ liệu user
```

3. KHÁI NIỆM GUARDED ROUTE

Guarded Route nghĩa là:

```
screen chỉ được truy cập
nếu user đã login
```

Nếu chưa login:

```
redirect → LoginScreen
```

4. LUỒNG HOẠT ĐỘNG

Luồng chuẩn:

```
User mở screen
↓
Check session
↓
Nếu login → cho phép truy cập
Nếu chưa login → redirect LoginScreen
```

5. KIẾN TRÚC ROUTE PROTECTION

Ứng dụng thường có:

```
LoginScreen
HomeScreen
ProfileScreen
SettingScreen
```

Các screen cần bảo vệ:

```
HomeScreen
ProfileScreen
SettingScreen
```

6. BƯỚC 1 – TẠO AUTH SERVICE

Tạo file:

```
lib/services/auth_service.dart
```

Code:

PhucPT10@fpt.edu.vn

```
import 'package:shared_preferences/shared_preferences.dart';

class AuthService {

  static Future<bool> isLoggedIn() async {

    final prefs = await SharedPreferences.getInstance();

    return prefs.getBool("isLoggedIn") ?? false;

  }

}
```

Giải thích

Hàm này dùng để:
kiểm tra trạng thái login

7. BƯỚC 2 – TẠO GUARD FUNCTION

Ta viết một hàm kiểm tra trước khi mở screen.
Ví dụ:

```
Future<void> openHome(BuildContext context) async {

  bool loggedIn = await AuthService.isLoggedIn();

  if(loggedIn){

    Navigator.push(
      context,
      MaterialPageRoute(builder: (_) => HomeScreen()),
    );

  }else{

    Navigator.push(
      context,
      MaterialPageRoute(builder: (_) => LoginScreen()),
    );

  }

}
```

8. GIẢI THÍCH GUARD

Logic:

```
check session
↓
true  → HomeScreen
false → LoginScreen
```

9. BƯỚC 3 – BẢO VỆ SCREEN

Ví dụ:
User bấm nút **Go to Profile**

PhucPT10@fpt.edu.vn

```
ElevatedButton(
  onPressed: () async {

    bool loggedIn = await AuthService.isLoggedIn();

    if(!loggedIn){

      Navigator.push(
        context,
        MaterialPageRoute(builder: (_) => LoginScreen()),
      );

      return;

    }

    Navigator.push(
      context,
      MaterialPageRoute(builder: (_) => ProfileScreen()),
    );

  },
  child: Text("Profile"),
)
```

10. BƯỚC 4 – AUTO CHECK KHI SCREEN LOAD

Một cách khác là kiểm tra **khi screen mở**.
Ví dụ trong `HomeScreen`.

```
@override
void initState() {
  super.initState();
  checkLogin();
}
```

Hàm checkLogin

```
void checkLogin() async {

  bool loggedIn = await AuthService.isLoggedIn();

  if(!loggedIn){

    Navigator.pushReplacement(
      context,
      MaterialPageRoute(builder: (_) => LoginScreen()),
    );

  }

}
```

11. LUỒNG HOẠT ĐỘNG THỰC TẾ

Ví dụ:

```
User login
↓
```

```
HomeScreen
↓
User logout
↓
User cố mở lại HomeScreen
↓
Guard check
↓
redirect LoginScreen
```

12. TEST KẾT QUẢ

Test các tình huống.

Case 1

```
User login
→ open Home
```

Kết quả:

```
HomeScreen hiển thị
```

Case 2

```
User chưa login
→ mở ProfileScreen
```

Kết quả:

```
redirect LoginScreen
```

Case 3

```
User logout
→ mở lại HomeScreen
```

Kết quả:

```
redirect LoginScreen
```

13. LỖI SINH VIÊN THƯỜNG GẶP

Lỗi 1

Không kiểm tra session.

```
Navigator.push(HomeScreen)
```

Lỗi 2

Logout nhưng không clear session.

```
isLoggedIn vẫn = true
```

Lỗi 3

Guard logic viết lặp lại nhiều nơi.

Nên dùng:

```
AuthService
```

14. BEST PRACTICES

1. Tách AuthService

Không viết logic login trực tiếp trong UI.

Ví dụ:

```
auth_service.dart
```


PhucPT10@fpt.edu.vn

2. Centralize Session

Chỉ dùng **1 nguồn session**

Ví dụ:

```
SharedPreferences
```

3. Guard tất cả protected screen

Ví dụ:

```
Home
Profile
Settings
Cart
Orders
```

15. TÓM TẮT KIẾN THỨC

Authentication trong app gồm 3 phần chính:

```
1 Login Form
2 Session Management
3 Guarded Routes
```

16. SƠ ĐỒ AUTH FLOW

```
User login
      ↓
Create session
      ↓
Navigate Home
      ↓
User mở screen
      ↓
Guard check session
      ↓
Allow / Redirect
```

17. MINI BÀI TẬP CHO SINH VIÊN

Hãy cải tiến ứng dụng:

1 bảo vệ **HomeScreen**

2 bảo vệ **ProfileScreen**

3 bảo vệ **FavoriteScreen**

Nếu user chưa login:

```
redirect LoginScreen
```


PhucPT10@fpt.edu.vn

V. KINH NGHIỆM THỰC TIỄN, TIPS

1. LOGIN FORM

Login form là **điểm bắt đầu của authentication flow**, nhưng nhiều sinh viên thường triển khai chưa đúng cách.

⚠ LỖI PHỔ BIẾN

✕ Không validate input

Sinh viên thường cho phép submit form dù dữ liệu chưa hợp lệ.
Ví dụ:

```
username = ""
password = ""
```

Nếu không kiểm tra trước khi submit:

hệ thống vẫn gửi request
gây lỗi authentication.

Trong Flutter, validation nên được thực hiện trong:

Form
TextFormField

✕ Password hiển thị rõ

Một lỗi phổ biến khác:
password được hiển thị trực tiếp trên màn hình.
Ví dụ:

```
password: 123456
```

Điều này gây ra **rủi ro bảo mật**.

✕ Không xử lý trạng thái loading

Một số app student:

```
nhấn Login → không có phản hồi
```

User không biết hệ thống đang xử lý.

BEST PRACTICES

✔ Validate form trước khi submit

Luôn kiểm tra dữ liệu:

```
username không được rỗng
password đủ độ dài
```

Flutter hỗ trợ validate trực tiếp trong:

Form

✔ Ẩn password bằng obscureText

Flutter hỗ trợ thuộc tính:

```
obscureText: true
```

trong **TextFormField**.
Điều này giúp:

```
password → ********
```

✔ Hiển thị lỗi rõ ràng

Ví dụ:

```
Invalid username or password
```

User phải biết:

login thất bại vì lý do gì.

PhucPT10@fpt.edu.vn

2. SESSION MANAGEMENT

Session management là **trung tâm của authentication system**.
Nhiều lỗi logic xảy ra ở phần này.

⚠ LỖI PHỔ BIẾN

✕ Không lưu session

Một số app chỉ:

```
login success → navigate Home
```

nhưng không lưu session.
Kết quả:

```
App restart → mất trạng thái login
```

User phải login lại.

✕ Logout không clear state

Một lỗi phổ biến khác:
User logout nhưng session vẫn còn.
Ví dụ:

```
logout → quay về Login
```

nhưng:

```
isLoggedIn = true
```

Kết quả:
user vẫn có thể truy cập screen protected.

✕ Session bị duplicate

Một số app lưu session ở nhiều nơi:

```
memory state
local storage
global variable
```

Điều này gây **inconsistent state**.

BEST PRACTICES

✔ Session phải có Single Source of Truth

Session chỉ nên được quản lý ở **một nơi duy nhất**.
Ví dụ:

```
AuthService
```

hoặc

```
SessionManager
```

✔ Luôn clear session khi logout

Logout phải thực hiện:

```
clear session
clear storage
redirect login
```

✔ Kiểm tra session khi app start

Khi app mở:

```
Check Session
↓
Logged In ? → Home
Logged Out → Login
```


PhucPT10@fpt.edu.vn

3. GUARDED ROUTES

Guarded routes giúp **bảo vệ các màn hình quan trọng**.

⚠ LỖI PHỔ BIẾN

✕ User có thể truy cập screen protected

Sinh viên thường viết:

```
Navigator.push(ProfileScreen);
```

nhưng không kiểm tra:

```
isLoggedIn
```

Kết quả:

user chưa login vẫn mở được screen.

✕ Không redirect khi session expired

Một số app không xử lý trường hợp:

```
session expired
```

User vẫn ở screen protected.

BEST PRACTICES

✔ Kiểm tra session trước navigation

Trước khi mở screen:

```
check isLoggedIn
```

Pseudo logic:

```
if (isLoggedIn)
    open Profile
else
    redirect Login
```

✔ Tạo Auth Guard

Trong ứng dụng lớn, developer thường tạo:

```
AuthGuard
```

hoặc

```
RouteMiddleware
```

để bảo vệ route.

✔ Centralize navigation logic

Không nên kiểm tra login ở nhiều nơi khác nhau.

Nên quản lý logic ở:

```
Navigation service
```

TIPS TỪ KINH NGHIỆM THỰC TẾ

Một số kinh nghiệm quan trọng khi xây login system:

Tip 1 – Authentication logic không nên nằm trong UI

Sai cách:

```
LoginScreen → xử lý toàn bộ auth logic
```

Đúng cách:

```
AuthService → xử lý authentication
```

Tip 2 – Session nên được quản lý global

Ví dụ:

```
AuthProvider
AuthController
```


PhucPT10@fpt.edu.vn

💡 Tip 3 – Luôn chuẩn bị cho logout

Logout phải:

```
clear token
clear session
clear cache
```

🎓 BÀI HỌC QUAN TRỌNG

Sinh viên cần nhớ **3 nguyên tắc quan trọng nhất của authentication system**.

1 Login chỉ là bước đầu

Authentication system gồm:

```
login
session
authorization
logout
```

2 Session giúp app nhớ trạng thái user

```
session = user identity state
```

3 Guarded routes giúp bảo vệ dữ liệu

Các screen quan trọng phải:

```
require authentication
```


PhucPT10@fpt.edu.vn

AUTH SESSION DEMO APP

1 TÓM TẮT DỰ ÁN

Tên dự án

Auth Session Demo App

Mục tiêu

Xây dựng ứng dụng Flutter có:

- Login screen (Form + validation)
- Mock authentication (tài khoản giả lập)
- Lưu session (đăng nhập bền vững khi restart app)
- Guarded routes (màn hình Profile chỉ mở khi login)
- Logout flow (xóa session + quay về login)

Sau dự án này sinh viên có thể:

- xây dựng login screen
- xử lý submit form
- thực hiện mock authentication
- lưu session state
- tạo guarded routes
- thực hiện logout flow

2 CHỨC NĂNG CỦA ỨNG DỤNG

Login

User nhập `username`, `password`

Hệ thống kiểm tra mock credential:

- **đúng**: `admin / 1234`
- **sai**: báo "Login failed"

Dashboard

Sau khi login thành công → chuyển sang Dashboard

Guarded Profile Screen

Profile chỉ mở khi đã login

Nếu chưa login → **redirect về Login**

Logout

Logout sẽ:

- clear session
- redirect về login
- không cho back lại màn protected

3 CÔNG NGHỆ SỬ DỤNG

- Flutter
- Dart
- `shared_preferences` (lưu session)

4 CẤU TRÚC THƯ MỤC DỰ ÁN

Code

```
lib/
│
├── main.dart
│
├── screens/
│   ├── login_screen.dart
│   ├── dashboard_screen.dart
│   └── profile_screen.dart
│
├── services/
│   ├── auth_service.dart
│   └── session_service.dart
│
└── guards/
    └── auth_guard.dart
```

5 TÓM TẮT QUY TRÌNH THỰC HIỆN (10 bước)

Create Flutter project
Add dependencies
Build Login UI
Form validation
Implement Mock Auth
Save session
Create Dashboard screen
Implement guarded route
Create Profile screen
Implement logout + check session at app start

6 STEP BY STEP IMPLEMENTATION (chi tiết từng thao tác)

STEP 1 — TẠO PROJECT

Mở Terminal / CMD:

```bash
flutter create auth_session_app
cd auth_session_app
code .
```

Kiểm tra chạy được app mặc định:

```bash
flutter run
```

STEP 2 — THÊM DEPENDENCY

Mở file **pubspec.yaml** → tìm `dependencies:` và thêm:

```YAML
dependencies:
  flutter:
    sdk: flutter
  shared_preferences: ^2.2.2
```

Chạy:

```bash
flutter pub get
```

Cách kiểm chứng: không báo lỗi, terminal có tải packages.

PhucPT10@fpt.edu.vn

STEP 3 — TẠO LOGIN SCREEN (UI cơ bản)
Tạo folder và file:
`lib/screens/login_screen.dart`
Dán code:
lib/screens/login_screen.dartv1

```
import 'package:flutter/material.dart';
import '../services/auth_service.dart';

class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key});

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  final _formKey = GlobalKey<FormState>();

  final usernameController = TextEditingController();
  final passwordController = TextEditingController();

  void login() async {
    if (_formKey.currentState!.validate()) {
      bool success = await AuthService.login(
        usernameController.text,
        passwordController.text,
      );

      if (success) {
        Navigator.pushReplacementNamed(context, "/dashboard");
      } else {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(content: Text("Login failed")),
        );
      }
    }
  }

  @override
  void dispose() {
    usernameController.dispose();
    passwordController.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Login")),
      body: Padding(
        padding: const EdgeInsets.all(20),
```


PhucPT10@fpt.edu.vn

```
        child: Form(
          key: _formKey,
          child: Column(
            children: [
              TextFormField(
                controller: usernameController,
                decoration: const InputDecoration(labelText: "Username"),
                validator: (value) {
                  if (value == null || value.isEmpty) {
                    return "Enter username";
                  }
                  return null;
                },
              ),
              TextFormField(
                controller: passwordController,
                obscureText: true,
                decoration: const InputDecoration(labelText: "Password"),
                validator: (value) {
                  if (value == null || value.length < 4) {
                    return "Password too short";
                  }
                  return null;
                },
              ),
              const SizedBox(height: 20),
              ElevatedButton(
                onPressed: login,
                child: const Text("Login"),
              )
            ],
          ),
        ),
      ),
    );
  }
}
```

Giải thích ngắn :

`Form` + `_formKey` giúp kiểm tra hợp lệ trước khi login.

`TextEditingController` để lấy text từ input.

`pushReplacementNamed` để user **không back về Login** sau khi đăng nhập.

Cách kiểm chứng:

Bấm Login khi chưa nhập username → hiện "Enter username"

Password < 4 ký tự → hiện "Password too short"

STEP 4 — MOCK AUTHENTICATION (AuthService)

Tạo file:

`lib/services/auth_service.dart`

lib/services/auth_service.dartv1

```
import 'session_service.dart';
```


PhucPT10@fpt.edu.vn

```
class AuthService {
  static Future<bool> login(String username, String password) async {
    if (username == "admin" && password == "1234") {
      await SessionService.saveSession(true);
      return true;
    }
    return false;
  }
}
```

Cách kiểm chứng:

Login `admin / 1234` → sẽ "được phép" (chút nữa ta cấu hình route để thấy dashboard)
Login sai → SnackBar "Login failed"

STEP 5 — SESSION SERVICE (shared_preferences)
Tạo file:
`lib/services/session_service.dart`

lib/services/session_service.dartv1

```
import 'package:shared_preferences/shared_preferences.dart';

class SessionService {
  static const String _key = "isLoggedIn";

  static Future<void> saveSession(bool value) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setBool(_key, value);
  }

  static Future<bool> isLoggedIn() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getBool(_key) ?? false;
  }

  static Future<void> clearSession() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.remove(_key);
  }
}
```

Cách kiểm chứng kỹ thuật session:

Nếu lưu `isLoggedIn=true`, tắt app mở lại mà vẫn vào dashboard (ở Step 10 sẽ làm).

STEP 6 — DASHBOARD SCREEN
Tạo file:
`lib/screens/dashboard_screen.dart`

lib/screens/dashboard_screen.dartv1

```
import 'package:flutter/material.dart';

class DashboardScreen extends StatelessWidget {
  const DashboardScreen({super.key});

  @override
```


PhucPT10@fpt.edu.vn

```
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Dashboard")),
      body: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          const Center(child: Text("Welcome to Dashboard")),
          const SizedBox(height: 12),
          ElevatedButton(
            onPressed: () {
              Navigator.pushNamed(context, "/profile");
            },
            child: const Text("Open Profile"),
          )
        ],
      ),
    );
  }
}
```

Cách kiểm chứng:

Sau login thành công → nhìn thấy dòng "Welcome to Dashboard"

Bấm "Open Profile" → sẽ đi profile (nhưng ta sẽ "guard" ở Step 8)

STEP 7 — PROFILE SCREEN + LOGOUT

Tạo file:

`lib/screens/profile_screen.dart`

lib/screens/profile_screen.dartv1

```
import 'package:flutter/material.dart';
import '../services/session_service.dart';

class ProfileScreen extends StatelessWidget {
  const ProfileScreen({super.key});

  void logout(BuildContext context) async {
    await SessionService.clearSession();
    Navigator.pushReplacementNamed(context, "/login");
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Profile")),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Text("User Profile"),
            const SizedBox(height: 12),
            ElevatedButton(
              onPressed: () => logout(context),
              child: const Text("Logout"),
            )
```

```
          ],
        ),
      ),
    );
  }
};
```

Cách kiểm chứng logout:

Bấm Logout → quay về Login

Nếu sau logout mà vẫn vào Profile được → nghĩa là guard chưa đúng (Step 8 sẽ xử lý)

STEP 8 — ROUTE GUARD (AuthGuard)

Tạo file:

`lib/guards/auth_guard.dart`

lib/guards/auth_guard.dartv1

```
import '../services/session_service.dart';

class AuthGuard {
  static Future<bool> check() async {
    return await SessionService.isLoggedIn();
  }
}
```

Lưu ý: Guard là "hỏi xem có login chưa".

STEP 9 — CẤU HÌNH ROUTES + GUARDED PROFILE ROUTE

Bây giờ ta làm phần quan trọng: **khi user mở `/profile` mà chưa login → ép về `/login`.**

Mở `lib/main.dart` và thay nội dung theo bản hoàn chỉnh dưới đây.

Điểm dễ sai của sinh viên: dùng `routes:` bình thường sẽ không chặn được `/profile`. Cách đơn giản cho bài này là dùng `onGenerateRoute` để "chặn" đường `/profile`.

lib/main.dartv1

```
import 'package:flutter/material.dart';

import 'guards/auth_guard.dart';
import 'screens/dashboard_screen.dart';
import 'screens/login_screen.dart';
import 'screens/profile_screen.dart';
import 'services/session_service.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final logged = await SessionService.isLoggedIn();
  runApp(MyApp(initialLoggedIn: logged));
}

class MyApp extends StatelessWidget {
  final bool initialLoggedIn;

  const MyApp({super.key, required this.initialLoggedIn});

  @override
```

```
Widget build(BuildContext context) {
  return MaterialApp(
    title: 'Auth Session Demo',
    debugShowCheckedModeBanner: false,
    initialRoute: initialLoggedIn ? "/dashboard" : "/login",
    onGenerateRoute: (settings) {
      // 1) Route thường
      if (settings.name == "/login") {
        return MaterialPageRoute(builder: (_) => const LoginScreen());
      }
      if (settings.name == "/dashboard") {
        return MaterialPageRoute(builder: (_) => const DashboardScreen());
      }

      // 2) Route cần guard
      if (settings.name == "/profile") {
        return MaterialPageRoute(
          builder: (_) {
            return FutureBuilder<bool>(
              future: AuthGuard.check(),
              builder: (context, snapshot) {
                // Loading state (đợi đọc SharedPreferences)
                if (!snapshot.hasData) {
                  return const Scaffold(
                    body: Center(child: CircularProgressIndicator()),
                  );
                }

                final ok = snapshot.data!;
                if (!ok) {
                  // Chưa login -> redirect login
                  // Dùng post-frame để tránh lỗi setState trong build
                  WidgetsBinding.instance.addPostFrameCallback((_) {
                    Navigator.pushReplacementNamed(context, "/login");
                  });

                  return const Scaffold(
                    body: Center(child: Text("Redirecting to Login...")),
                  );
                }

                return const ProfileScreen();
              },
            );
          },
        );
      }

      // 3) Unknown route fallback
      return MaterialPageRoute(
        builder: (_) => const Scaffold(
          body: Center(child: Text("404 - Route not found")),
        ),
```


PhucPT10@fpt.edu.vn

```
        );
      },
    );
  }
}
```

Cách kiểm chứng Guarded Routes (rất quan trọng):

Chạy app, **chưa login**

(Trên một số cách test) nếu bạn điều hướng tới `/profile` (ví dụ sau này gõ deep link / hoặc code pushNamed)

→ Expected: **bị đẩy về Login**

Login đúng → vào dashboard → bấm Open Profile → Expected: Profile hiển thị

STEP 10 — KIỂM TRA SESSION KHI APP START (đã làm ở main)

Trong code `main.dart` phía trên đã có:

```
WidgetsFlutterBinding.ensureInitialized()
final logged = await SessionService.isLoggedIn();
initialRoute: initialLoggedIn ? "/dashboard" : "/login"
```

Cách kiểm chứng session persistence:

Login đúng → vào Dashboard

Tắt app hoàn toàn (stop) → chạy lại

Expected: app **vào thẳng Dashboard** (vì session còn)

7 KIỂM CHỨNG KẾT QUẢ (Checklist test)

TEST 1 — Login đúng

Input:

username: `admin`

password: `1234`

Expected:

→ Dashboard

TEST 2 — Login sai

Expected:

SnackBar "Login failed"

TEST 3 — Mở Profile khi đã login

Action:

Dashboard → Open Profile

Expected:

Profile screen hiển thị

TEST 4 — Logout

Action:

Profile → Logout

Expected:

→ Login screen

Không được back về Profile

TEST 5 — Restart App

Expected:

Nếu chưa logout → vẫn login (vào Dashboard)

Nếu đã logout → vào Login

PhucPT10@fpt.edu.vn

8 LỖI PHỔ BIẾN (và cách tránh)

Quên `await SharedPreferences.getInstance()`
Hậu quả: session không lưu/đọc đúng
Cách tránh: mọi hàm session đều `async` và có `await`

Không dùng `pushReplacement...`
Hậu quả: back quay về màn trước (vd dashboard back ra login)
Cách tránh: login success và logout nên dùng `pushReplacementNamed`

Chưa guard route `/profile`
Hậu quả: vẫn vào Profile dù chưa login
Cách tránh: chặn ở `onGenerateRoute` như tài liệu

9 MẸO QUAN TRỌNG (chuẩn hóa tư duy)

Tách `AuthService` và `SessionService`:
- UI chỉ gọi service
- Service xử lý logic
- Dễ sửa, dễ test, dễ mở rộng (API thật sau này)

Không đặt logic auth trong UI:
- Sai: LoginScreen tự set SharedPreferences
- Đúng: LoginScreen → gọi `AuthService.login()`

10 TỔNG HỢP ĐẦY ĐỦ CODE (final project)

pubspec.yaml (phần dependency)

pubspec.yaml

```yaml
dependencies:
  flutter:
    sdk: flutter
  shared_preferences: ^2.2.2
```

lib/main.dart

lib/main.dartv2

```dart
import 'package:flutter/material.dart';

import 'guards/auth_guard.dart';
import 'screens/dashboard_screen.dart';
import 'screens/login_screen.dart';
import 'screens/profile_screen.dart';
import 'services/session_service.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final logged = await SessionService.isLoggedIn();
  runApp(MyApp(initialLoggedIn: logged));
}

class MyApp extends StatelessWidget {
  final bool initialLoggedIn;

  const MyApp({super.key, required this.initialLoggedIn});
```


PhucPT10@fpt.edu.vn

```
@override
Widget build(BuildContext context) {
  return MaterialApp(
    title: 'Auth Session Demo',
    debugShowCheckedModeBanner: false,
    initialRoute: initialLoggedIn ? "/dashboard" : "/login",
    onGenerateRoute: (settings) {
      if (settings.name == "/login") {
        return MaterialPageRoute(builder: (_) => const LoginScreen());
      }
      if (settings.name == "/dashboard") {
        return MaterialPageRoute(builder: (_) => const DashboardScreen());
      }

      if (settings.name == "/profile") {
        return MaterialPageRoute(
          builder: (_) {
            return FutureBuilder<bool>(
              future: AuthGuard.check(),
              builder: (context, snapshot) {
                if (!snapshot.hasData) {
                  return const Scaffold(
                    body: Center(child: CircularProgressIndicator()),
                  );
                }

                final ok = snapshot.data!;
                if (!ok) {
                  WidgetsBinding.instance.addPostFrameCallback((_) {
                    Navigator.pushReplacementNamed(context, "/login");
                  });

                  return const Scaffold(
                    body: Center(child: Text("Redirecting to Login...")),
                  );
                }

                return const ProfileScreen();
              },
            );
          },
        );
      }

      return MaterialPageRoute(
        builder: (_) => const Scaffold(
          body: Center(child: Text("404 - Route not found")),
        ),
      );
    },
  );
}
```


PhucPT10@fpt.edu.vn

```
}
```

lib/screens/login_screen.dart

lib/screens/login_screen.dartv2

```
import 'package:flutter/material.dart';
import '../services/auth_service.dart';

class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key});

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  final _formKey = GlobalKey<FormState>();

  final usernameController = TextEditingController();
  final passwordController = TextEditingController();

  void login() async {
    if (_formKey.currentState!.validate()) {
      bool success = await AuthService.login(
        usernameController.text,
        passwordController.text,
      );

      if (success) {
        Navigator.pushReplacementNamed(context, "/dashboard");
      } else {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(content: Text("Login failed")),
        );
      }
    }
  }

  @override
  void dispose() {
    usernameController.dispose();
    passwordController.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Login")),
      body: Padding(
        padding: const EdgeInsets.all(20),
        child: Form(
```

```
          key: _formKey,
          child: Column(
            children: [
              TextFormField(
                controller: usernameController,
                decoration: const InputDecoration(labelText: "Username"),
                validator: (value) {
                  if (value == null || value.isEmpty) {
                    return "Enter username";
                  }
                  return null;
                },
              ),
              TextFormField(
                controller: passwordController,
                obscureText: true,
                decoration: const InputDecoration(labelText: "Password"),
                validator: (value) {
                  if (value == null || value.length < 4) {
                    return "Password too short";
                  }
                  return null;
                },
              ),
              const SizedBox(height: 20),
              ElevatedButton(
                onPressed: login,
                child: const Text("Login"),
              )
            ],
          ),
        ),
      ),
    );
  }
}
```

lib/screens/dashboard_screen.dart

lib/screens/dashboard_screen.dartv2

PhucPT10@fpt.edu.vn

```
import 'package:flutter/material.dart';

class DashboardScreen extends StatelessWidget {
  const DashboardScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Dashboard")),
      body: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          const Center(child: Text("Welcome to Dashboard")),
          const SizedBox(height: 12),
          ElevatedButton(
            onPressed: () {
              Navigator.pushNamed(context, "/profile");
            },
            child: const Text("Open Profile"),
          )
        ],
      ),
    );
  }
}
```

lib/screens/profile_screen.dart

lib/screens/profile_screen.dartv2

```
import 'package:flutter/material.dart';
import '../services/session_service.dart';

class ProfileScreen extends StatelessWidget {
  const ProfileScreen({super.key});

  void logout(BuildContext context) async {
    await SessionService.clearSession();
    Navigator.pushReplacementNamed(context, "/login");
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Profile")),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Text("User Profile"),
            const SizedBox(height: 12),
            ElevatedButton(
              onPressed: () => logout(context),
              child: const Text("Logout"),
```

```
            )
          ],
        ),
      ),
    );
  }
}
```

lib/services/auth_service.dart

lib/services/auth_service.dartv2

```
import 'session_service.dart';

class AuthService {
  static Future<bool> login(String username, String password) async {
    if (username == "admin" && password == "1234") {
      await SessionService.saveSession(true);
      return true;
    }
    return false;
  }
}
```

lib/services/session_service.dart

lib/services/session_service.dartv2

```
import 'package:shared_preferences/shared_preferences.dart';

class SessionService {
  static const String _key = "isLoggedIn";

  static Future<void> saveSession(bool value) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setBool(_key, value);
  }

  static Future<bool> isLoggedIn() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getBool(_key) ?? false;
  }

  static Future<void> clearSession() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.remove(_key);
  }
}
```

lib/guards/auth_guard.dart

lib/guards/auth_guard.dartv2

```
import '../services/session_service.dart';

class AuthGuard {
```

```
  static Future<bool> check() async {
    return await SessionService.isLoggedIn();
  }
}
```

1 1 "Phiên bản cập nhật theo từng bước code" (để sinh viên theo dõi tiến trình)
Để bạn đưa vào tài liệu dạng "timeline", bạn có thể mô tả các mốc thay đổi như sau:

- **V0 (sau Step 2):** Chỉ thêm dependency `shared_preferences`
- **V1 (sau Step 3):** Có `LoginScreen` + form validation (chưa hoạt động hoàn chỉnh vì chưa có routes/main)
- **V2 (sau Step 5):** Có `AuthService` + `SessionService` (login bắt đầu lưu session)
- **V3 (sau Step 7):** Có đủ `DashboardScreen`, `ProfileScreen` (đã có logout)
- **V4 (sau Step 9–10):** `main.dart` dùng `onGenerateRoute` + guard `/profile` + initialRoute theo

PhucPT10@fpt.edu.vn

TỪ ĐIỂN THUẬT NGỮ (FOUNDATION GLOSSARY)

Thuật ngữ	Định nghĩa	Ví dụ trong ứng dụng
Authentication	Quá trình xác thực danh tính người dùng khi truy cập hệ thống.	User nhập username/password để login.
Authorization	Quá trình kiểm soát quyền truy cập sau khi user đã được xác thực.	Chỉ admin mới mở được Admin Screen.
Login	Hành động người dùng nhập thông tin xác thực để truy cập hệ thống.	Login bằng email và password.
Logout	Hành động kết thúc phiên làm việc của người dùng.	User nhấn Logout để thoát khỏi ứng dụng.
Credential	Thông tin xác thực của người dùng như username và password.	admin / 1234
Login Form	Form giao diện để người dùng nhập thông tin đăng nhập.	Form gồm email và password.
Form Validation	Quá trình kiểm tra dữ liệu nhập vào trước khi gửi đi xử lý.	Password phải có ít nhất 6 ký tự.
Mock Authentication	Cách giả lập hệ thống login mà không cần server thật.	Hard-code username/password trong app.
Authentication Flow	Chuỗi các bước hệ thống thực hiện để xác thực user.	Input → Validate → Verify → Create Session
Session	Trạng thái đăng nhập của user trong ứng dụng.	isLoggedIn = true
Session State	Dữ liệu mô tả trạng thái login của user.	User đã login hay chưa.
Session Persistence	Khả năng lưu session để user không cần login lại khi mở app.	Lưu trạng thái login trong local storage.
Local Storage	Cơ chế lưu dữ liệu trên thiết bị của người dùng.	Lưu session trong storage.
Shared Preferences	Plugin lưu dữ liệu key–value đơn giản trong Flutter.	Lưu biến isLoggedIn.
Guarded Route	Route chỉ cho phép truy cập khi thỏa điều kiện authentication.	Profile screen chỉ mở khi login.
Route Guard	Logic kiểm tra điều kiện trước khi cho phép truy cập route.	Nếu chưa login → redirect login.
Protected Screen	Màn hình yêu cầu user phải login mới được truy cập.	Profile screen.
Navigation	Quá trình chuyển từ screen này sang screen khác.	Login → Dashboard.
Redirect	Chuyển hướng user sang screen khác khi điều kiện không thỏa.	User chưa login → redirect LoginScreen.
Session Check	Quá trình kiểm tra trạng thái login khi app khởi động.	Kiểm tra isLoggedIn khi mở app.
Single Source of Truth	Nguyên tắc chỉ có một nơi quản lý dữ liệu trạng thái.	Session chỉ được quản lý bởi SessionService.
Auth Service	Lớp chịu trách nhiệm xử lý logic authentication.	login(), logout()
Session Service	Lớp quản lý trạng thái session của user.	saveSession(), clearSession()
State Management	Cách quản lý trạng thái dữ liệu trong ứng dụng.	Quản lý trạng thái login.
Secure Storage	Cơ chế lưu dữ liệu nhạy cảm một cách an toàn.	Lưu token login trong storage an toàn.

PhucPT10@fpt.edu.vn

10 PROMPT TỰ NGHIÊN CỨU CHUYÊN SÂU

#	Prompt nghiên cứu
1	Phân tích sự khác biệt giữa **in-memory state** và **persistent state** trong ứng dụng mobile. Khi nào nên dùng mỗi loại?
2	Giải thích cách **state lifecycle** hoạt động trong Flutter và ảnh hưởng của nó đến việc quản lý dữ liệu trong ứng dụng.
3	Phân tích lợi ích và hạn chế của việc lưu dữ liệu bằng **shared_preferences** so với database local.
4	Trong Flutter, tại sao việc tách **data layer** và **UI layer** lại quan trọng đối với khả năng mở rộng ứng dụng?
5	Phân tích cách **state synchronization** hoạt động khi nhiều widget cùng phụ thuộc vào một nguồn dữ liệu.
6	So sánh cách quản lý state bằng **local widget state** và **global state management**.
7	Phân tích tác động của việc **rebuild widget tree** đối với hiệu năng khi dữ liệu thay đổi.
8	Giải thích tại sao việc thiết kế **data model rõ ràng** lại giúp giảm bug trong ứng dụng mobile.
9	Phân tích vai trò của **serialization (JSON encode/decode)** trong việc lưu trữ dữ liệu persistent.
10	Đánh giá các tiêu chí lựa chọn giải pháp lưu trữ dữ liệu local trong mobile app (simple storage vs local database).

PhucPT10@fpt.edu.vn

10 PROMPT TỰ NGHIÊN CỨU THỰC TIỄN CHUYÊN NGHIỆP

#	Prompt nghiên cứu
1	Trong ứng dụng mobile lớn, developer thường tổ chức **data layer** như thế nào để dễ bảo trì và mở rộng?
2	Khi ứng dụng cần lưu nhiều dữ liệu offline, tại sao các công ty thường chuyển từ **shared_preferences** sang database local như **SQLite**?
3	Làm thế nào để đảm bảo dữ liệu local không bị mất khi ứng dụng bị crash hoặc bị kill bởi hệ điều hành?
4	Trong ứng dụng production, developer xử lý **data migration** như thế nào khi cấu trúc dữ liệu thay đổi?
5	Khi xây dựng ứng dụng có offline mode, cách đồng bộ dữ liệu giữa local storage và server được thiết kế ra sao?
6	Những vấn đề bảo mật nào cần được xem xét khi lưu dữ liệu người dùng trên thiết bị mobile?
7	Khi ứng dụng có nhiều màn hình sử dụng cùng một dữ liệu, làm thế nào để tránh **data inconsistency**?
8	Trong team phát triển lớn, làm thế nào để đảm bảo các developer tuân thủ cùng một **data architecture pattern**?
9	Khi dữ liệu local trở nên lớn (hàng nghìn records), chiến lược tối ưu hiệu năng đọc/ghi dữ liệu là gì?
10	Trong ứng dụng thực tế, làm thế nào để debug các lỗi liên quan đến **state synchronization**?
11	Khi ứng dụng cần export dữ liệu ra file (ví dụ JSON backup), những yếu tố UX nào cần được xem xét?
12	Làm thế nào để đảm bảo dữ liệu backup có thể **import lại an toàn** trong các phiên bản app khác nhau?
13	Khi nhiều người dùng cùng sử dụng ứng dụng trên nhiều thiết bị, làm thế nào để tránh xung đột dữ liệu?
14	Khi thiết kế hệ thống lưu trữ dữ liệu cho ứng dụng mobile, các công ty thường áp dụng **clean architecture** như thế nào?
15	Phân tích các chiến lược test (unit test / integration test) để đảm bảo hệ thống quản lý dữ liệu hoạt động ổn định.

PhucPT10@fpt.edu.vn